MPC-PiNNs technique applied to Quadrotors

Elective in Robotics [UAV] Course Project

Master's Degree in Artificial Intelligence and Robotics

Bugli Eugenio, Di Marzo Gabriele, Maiorana Flavio

Academic Year 2024/2025

SAPIENZA
UNIVERSITÀ DI ROMA

Table of Contents

Neural Model Predictive Control (NMPC)

1 Introduction

In this work we have tried to combine Model Predictive Control along with PiNNs to trajectory tracking problem applied to quadrotors. We have used the Unicycle robot as simple test case for our experiments.

Table of Contents

2 Quadrotor Model

Quadrotor Standard Model

Starting from the Translational (1) and Rotational Dynamics (2), we have derived the simplified second order of the quadrotor.

$$m\ddot{p} = f_t + f_g + f_a + f_d \tag{1}$$

$$J\dot{\omega} + \omega \times (J\omega) = \tau \tag{2}$$

$$\begin{cases} \ddot{x} = -\left(\cos\psi\ \sin\theta\ \cos\phi + \sin\psi\ \sin\phi\right)\frac{T}{m}, \\ \ddot{y} = -\left(\sin\psi\ \sin\theta\ \cos\phi - \cos\psi\ \sin\phi\right)\frac{T}{m}, \\ \ddot{z} = -\left(\cos\theta\ \cos\phi\right)\frac{T}{m} + g, \\ \ddot{\phi} = \frac{\tau_\phi}{I_x} \\ \ddot{\theta} = \frac{\tau_\theta}{I_y} \\ \ddot{\psi} = \frac{\tau_\psi}{I_z} \end{cases} \tag{3}$$

Crazyflie Model

2 Quadrotor Model

Since we have used the Crazyflie as reference, the reference model is different with respect to the previous one. This is a consequence of how the firmware of the robot acts. There is a low level attitude controller that takes as inputs the RPY desired poses and maps them as control torques.

$$
\begin{cases}
\dot{x} = v_x, \\
\dot{y} = v_y, \\
\dot{z} = v_z, \\
\dot{v}_x = -\Big(\cos\psi\ \sin\theta\ \cos\phi + \sin\psi\ \sin\phi\Big)\frac{T}{m}, \\
\dot{v}_y = -\Big(\sin\psi\ \sin\theta\ \cos\phi - \cos\psi\ \sin\phi\Big)\frac{T}{m}, \\
\dot{v}_z = -\Big(\cos\theta\ \cos\phi\Big)\dfrac{T}{m} + g, \\
\dot{\phi} = (\phi_c - \phi)/\tau \\
\dot{\theta} = (\theta_c - \theta)/\tau \\
\dot{\psi} = (\psi_c - \psi)/\tau
\end{cases}
\tag{4}
$$

Table of Contents

3 Model Predictive Control

MPC Structure

3 Model Predictive Control

We have used for both robots the following structure for the MPC:

$$\min_{u(0),u(1),\ldots,u(N-1)} \quad \sum_{k=0}^{N-1} \ell(x(k),u(k)) + \ell_f(x(N)) \tag{5}$$

$$\text{s.t.} \quad x(k+1) = f(x(k),u(k)), \quad k = 0,\ldots,N-1, \tag{6}$$

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}, \quad k = 0,\ldots,N-1, \tag{7}$$

$$x(0) = x_{0.} \tag{8}$$

MPC Cost Function

Cost Function

$$\ell(x(k), u(k)) = \sum_{k=0}^{N-1} \Big[\sum_{x \in \mathcal{X}} W_{x(k)} \Big(x(k)_{\text{ref}} - x(k) \Big)^2 + \sum_{u \in \mathcal{U}} W_u \, u^2 \Big],$$

$$\ell_f(x(N)) = \sum_{x \in \mathcal{X}} W_{x(N)} \Big(x(N)_{\text{ref}} - x(N) \Big)^2.$$

Unicycle State & Control

$$\begin{cases} \mathcal{X} = \begin{bmatrix} x & y & \theta \end{bmatrix}^T, \\ \mathcal{U} = \begin{bmatrix} v & \omega \end{bmatrix}^T. \end{cases}$$

Quadrotor State & Control

$$\begin{cases} \mathcal{X} = \begin{bmatrix} x & y & z & v_x & v_y & v_z & \phi & \theta & \psi \end{bmatrix}^T, \\ \mathcal{U} = \begin{bmatrix} \phi_c & \theta_c & \psi_c & T \end{bmatrix}^T. \end{cases}$$

Table of Contents

4 Dataset

Structure

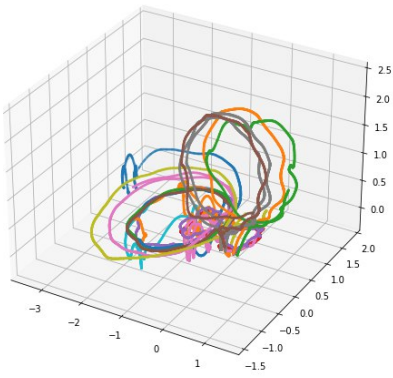
4 Dataset

The data the we used to train our neural models come from simulation and some real world experiments.

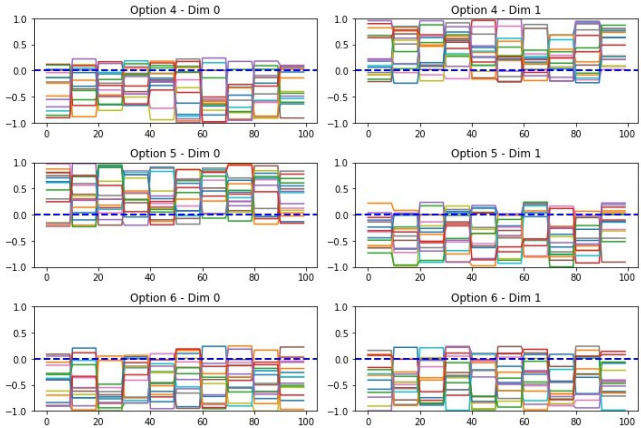
- Generally speaking, we can describe each our dataset as a tuple in the form $\{[x_t, u_t], [x_{t+1}]\}$, were x_t is the state of the system at time t, u_t is the control input at time t and x_{t+1} is the state of the system at time $t + 1$.
- For the baseline (*simple-neural-network*), no constraints on the dataset form are required, while on the physics informed models the input needs to be **constant** in a sub-interval $[0, T]$.

Examples

4 Dataset



Quadrotor data



Unicycle data (PiNN)

Problems

4 Dataset

When extracting data from the real-world experiments (or the gazebo simulations), state and action **publishing rates** were a crucial aspect to take into account.

- The action publishing rate should be high enough to make the controller safe enough
- The state publishing rate should be high, but it couldn't be too high, for communication latency reasons, which would have made the use of the real crazyflie impossible

These problems could be **partially mitigated** by using the gazebo simulator and setting ad-hoc publishing rates, or forcing the input to be constant (introducing a noisy approximation) for a certain amount of time in the real-world data.

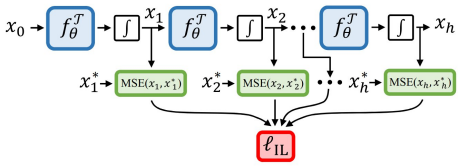
Table of Contents

Training Scheme

5 Neural Network

The multi-step training procedure is defined as follows:

- Give in input the pair (s_t, a_t) to the network
- Perform the integration of the output in the sample time dt
- Add what we have to the input to obtain the new state s_{t+1}
- Compute the **MSE** loss
- Use the new state as the next input of the network



Unicycle Network

5 Neural Network

The neural network used to model the kinematic model of the unicycle is a simple MLP architecture with two hidden layers of 128 neurons. The input is given by the concatenation of the current state and the current control inputs:

$$X = [x, y, \theta, v, \omega] \tag{9}$$

while the output represents the dynamics of the system:

$$Y = [\dot{x}, \dot{y}, \dot{\theta}] \tag{10}$$

Quadrotor Network

5 Neural Network

The quadrotor dynamics presents a more complex model, with a state $x \in R^9$ and control inputs $u \in R^4$. For approximating such dynamics with the defined training scheme, a larger network has been tested: we have increased the number of hidden layers up to 3 and the latent dimension from 128 to 256. However, the main problem during training is related to the generation of the data.

$$X = \begin{bmatrix} x & y & z & v_x & v_y & v_z & \phi & \theta & \psi \mid U \end{bmatrix}^T \tag{11}$$

$$U = \begin{bmatrix} \phi_c & \theta_c & \psi_c & T \end{bmatrix}^T \tag{12}$$

$$Y = \begin{bmatrix} \dot{x} & \dot{y} & \dot{z} & \dot{v_x} & \dot{v_y} & \dot{v_z} & \dot{\phi} & \dot{\theta} & \dot{\psi} \end{bmatrix}^T \tag{13}$$

Table of Contents

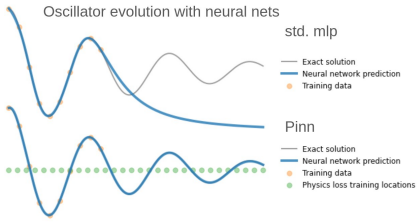
PiNNs

6 Physical Informed Neural Networks

Physical Informed Neural Networks (PiNNs) propose an alternative approach where the solution is approximated by a neural network that is trained not only on data but also by enforcing the underlying physical laws via the differential equations.

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_u} \sum_{i=1}^{N_u} \left| u_\theta(\mathbf{x}_i, t_i) - u_i^{\text{obs}} \right|^2, \quad (14)$$

$$\mathcal{L}(\theta)_{physics} = \frac{1}{N} \sum_{i=1}^{N} (\nabla_x - ode(x, u, t))^2, \quad (15)$$



PiNN vs Classical MLP

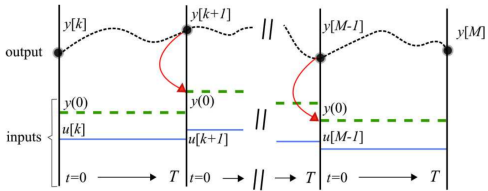
Problems and Limitations:

- Cannot handle control input
- Fixed initial condition

PiNN with Controls

6 Physical Informed Neural Networks

The Pinn-C modifies the formulation of the pinn, by dividing the evolution into sub-transitions with constant input. The network takes as input $(x(0), \bar{u}, t)$.



Pros:

- it can handle larger time-steps with respect to the original formulation
- can accept arbitrary starting conditions

Cons:

- relies heavily on the dataset

Training Scheme

Training Loop: Loss Calculation

1: **Input:** X_{data}, y
2: **Initialize:** $L \leftarrow 0$
3: **Compute Predictions:** $\hat{y} \leftarrow \mathcal{M}(X_{\text{data}})$
4: **Update Loss with MSE:** $L \leftarrow L + \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
5: **Sample Input Data:** $X \sim \mathcal{D}$
6: **Calculate Gradient:** $J \leftarrow \nabla_X \theta$
7: **Compute Analytical ODE:** $\text{ode}(X) = f(X)$
8: **Update Loss with ODE Term:** $L \leftarrow L + \frac{1}{n}\sum_{i=1}^{n}\left(\text{ode}(X) - \nabla_X \theta\right)^2$
9: **Sample a Point:** $X_{\text{point}} \sim \mathcal{D}$
10: **Set to zero the Input:** $X_{\text{input}} \leftarrow 0$
11: **Update Loss with Boundary Term:** $L \leftarrow L + \frac{1}{n}\sum_{i=1}^{n}(X_i - \hat{y}_i)^2$
12: **Output:** Updated loss L

Table of Contents

7 Results and Problems Encountered

Dataset Generation

7 Results and Problems Encountered

We have tried different strategies:

- random velocity commands over the horizon
- random velocity commands constant over the horizon

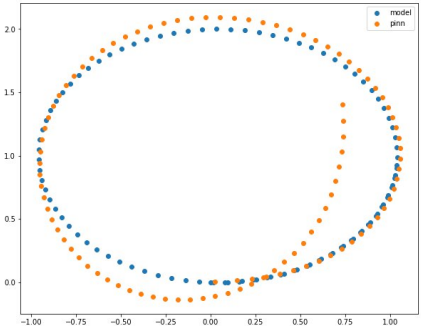
The system has been integrated with Euler or Runge-Kutta 4.

Drift Problem

7 Results and Problems Encountered

After some training, we have tested our neural model along with the MPC with nice results (check video). However, we found that by having zero control inputs the network still moves the state somewhere else. We have tried to overcome this problem in some ways but with poor results:

- Sample N points at the boundary during each epoch
- Add a penalty term related to the boundary condition

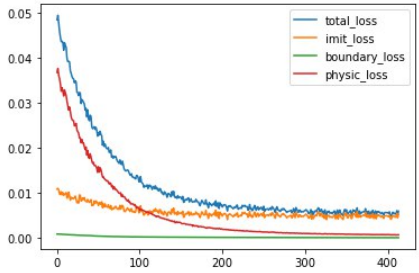


PiNN-C vs Reference

Possible Local Minima

7 Results and Problems Encountered

During each training experiment, we have found that all losses behave correctly but they remain stuck at some points even with few iterations. Also in this case we have tried extensively hyperparameter tuning without any solution to the problem.



Loss Behavior

MPC-PiNNs technique applied to Quadrotors

Thank you for listening